



Name: Mohsin Ali

Roll Number: 25P-0545

Section: BCS-3B

Course: Data Structure-LAB

Question 1. Pointer-Indexed Array Analysis: Your Weekly Activity Log

Step 1:

Number of Steps Each Day of the Week

Days	Step
Monday	9000
Tuesday	12000
Wednesday	10000
Thursday	15000
Friday	13000
Saturday	12000
Sunday	6000

Step 2:

```
#include <iostream>
using namespace std;
void analyzeArray(int* arr, int size) {
    int* p = arr;
    int total = 0;
    for (int i = 0; i<size; i++) {
        total += *p;
        p++;
    }
    int average = total / size;
    int* maxPtr = arr;
    p = arr;
    for (int i = 0; i<size; i++) {
        if (*p>*maxPtr) {
            maxPtr = p;
        }
        p++;
    }
    cout<<"Total: "<<total<<endl;
    cout<<"Average: "<<average<<endl;
    cout<<"MAXIMUM NUmber: "<<*(maxPtr)<<" Addresss: "<<maxPtr<<endl;
```

```

}
int main(){
int *arr=new int[7];
arr[0]=9000;
arr[1]=12000;
arr[2]=10000;
arr[3]=15000;
arr[4]=12000;
arr[5]=13000;
arr[6]=6000;
analyzeArray(arr,7);
}

```

The screenshot shows a C++ IDE with a file named `ltask1_lab1.cpp`. The code defines a function `analyzeArray` that takes an array of integers and its size. The function calculates the total sum of the array elements, the average, and the maximum value along with its memory address. The `main` function initializes an array with seven values and calls `analyzeArray`.

```

1  2
3  #include <iostream>
4
5  using namespace std;
6
7
8  void analyzeArray(int* arr, int size) {
9      int* p = arr;
10     int total = 0;
11     for (int i = 0; i<size; i++) {
12         total += *p;
13         p++;
14     }
15     int average = total / size;
16     int* maxPtr = arr;
17     p = arr;
18     for (int i = 0; i<size; i++) {
19         if (*p>*maxPtr) {
20             maxPtr = p;
21         }
22         p++;
23     }
24
25
26     cout<<"Total: "<<total<<endl;
27     cout<<"Average: "<<average<<endl;
28
29     cout<<"MAXIMUM Number: "<<*(maxPtr)<<" Addresss: "<<maxPtr<<endl;
30
31

```

The terminal output shows the results of the program execution:

```

ali@fedora:~/run/media/ali/New Volume/Work Space/University/FAST/Classes/3rd_Sems/DSA/Classes$ ./ltask1_lab1
Total: 77000
Average: 11000
MAXIMUM Number: 15000 Addresss: 0x1277232c
ali@fedora:~/run/media/ali/New Volume/Work Space/University/FAST/Classes/3rd_Sems/DSA/Classes$

```

Reasoning: after the first loop calculates the total of the array elements, the program find the average of it and then we also initialize the maxptr to arr starting index and also reset p pointer to starting index of the array, after that new loop goes through all the indexes of the pointers and checks which elements holds the maximum number and then store its address into maxptr,

Step 3:

Got the correct and matching result from the code and the manually dry running
Total: 77000, Average : 11000, MaxPtr= arr+3 = 15000

The image shows handwritten notes on lined paper, divided into two main sections. The left section contains code snippets and a table for a dry run. The right section contains a table for finding the maximum pointer in an array.

Left Section:

Code snippets:

```
#(curr + 1) = val;
len++;
}
```

DS-LAB :: Assignment Lab-1

Step-1

Q#1: Steps per day

Day	Steps
Day 1	9000
Day 2	12000
Day 3	10000
Day 4	15000
Day 5	12000
Day 6	13000
Day 7	6000

Step-3

// For Total and Average

Initially: $p = \text{arr} \rightarrow \text{arr}[0]$

total = 0;

It	*p	Sum	Total
0	9000	0 + 9000	9000
1	12000	9000 + 12000	21000
2	10000	21000 + 10000	31000
3	15000	31000 + 15000	46000
4	12000	46000 + 12000	58000
5	13000	58000 + 13000	71000
6	6000	71000 + 6000	77000

Right Section:

// For maxPtr:

Again resetting:

$\text{maxPtr} = \text{arr} \rightarrow \text{arr}[0]$

$p = \text{arr} \rightarrow \text{arr}[0]$

It	*p	*maxPtr before	*p > *maxPtr	*maxPtr
0	9000	9000	False	arr+0
1	12000	9000	True	arr+1
2	10000	12000	False	arr+1
3	15000	12000	True	arr+3
4	12000	15000	False	arr+3
5	13000	15000	False	arr+3
6	6000	15000	False	arr+3

Thus: $\text{maxPtr} = \text{arr}+3;$

$\text{maxPtr} = 15000$

Step 4: Adversarial Analysis :

A realistic edge case that would break the function is an empty array with $\text{size} = 0$. For example, the input could be arr with zero elements and $\text{size} = 0$. The first loop would not

execute, but the function would then calculate $\text{average} = \text{total} / \text{size}$, resulting in division by zero. Furthermore, `maxPtr` is initialized to `arr` even though there is no valid element, and the function later dereferences `maxPtr`, which is also invalid. Therefore, the function should check whether `size` is greater than zero before performing these operations.

Question 2. Pointer-Based Array Iterator Design

Step1:

```
#include <iostream>
using namespace std;
```

```
void chk(char* a,int n,int ti){
if(ti<0 || ti>=n){
cout<<"Invalid index";
return;
}
```

```
char* p=a+ti;
char c=*p;
```

```
cout<<"Char: "<<c<<endl;
```

```
if(c=='a' || c=='e' || c=='i' || c=='o' || c=='u' ||
c=='A' || c=='E' || c=='I' || c=='O' || c=='U')
cout<<"Vowel"<<endl;
else
cout<<"Not Vowel"<<endl;
```

```
cout<<"Address: "<<(void*)p<<endl;
}
```

```
int main(){
char nm[]="MohsinAli";
// char *nm;
// // nm=new char[100]; //does work but ISO c++ forbids string constant to char* so we using direct
char nm[] for this one only
// nm="MohsinAli";
chk(nm,9,0);
return 0;
}
```

ltask2_lab1.cpp > chk(char *, int, int)

```
1  #include <iostream>
2  using namespace std;
3
4  void chk(char* a,int n,int ti){
5      if(ti<0||ti>=n){
6          cout<<"Invalid index";
7          return;
8      }
9
10     char* p=a+ti;
11     char c=*p;
12
13     cout<<"Char: "<<c<<endl;
14
15     if(c=='a' || c=='e' || c=='i' || c=='o' || c=='u' ||
16        c=='A' || c=='E' || c=='I' || c=='O' || c=='U')
17         cout<<"Vowel"<<endl;
18     else
19         cout<<"Not Vowel"<<endl;
20
21     cout<<"Address: "<<(void*)p<<endl;
22 }
23
24 int main(){
25     char nm[]="MohsinAli";
26     chk(nm,9,4);
27     return 0;
28 }
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
ali@fedora:/run/media/ali/New Volume/Work Space/University/FAST/Classes/3rd_Sems/DSA/Classes$ "/run/media/ali/New
k2_lab1"
Char: i
Vowel
Address: 0x7ffdf86bfaa
ali@fedora:/run/media/ali/New Volume/Work Space/University/FAST/Classes/3rd_Sems/DSA/Classes$
```

Step 2:

Pointer arithmetic is appropriate because the array is passed as a raw pointer, so the pointer directly represents the starting address of the array. Using `a+ti` allows us to move to the required element and access it with `*`, without using array indexing.

Step3:

Suppose the array is "Ali" but the size is mistakenly given as 10 and targetIndex is 8. The actual valid indexes are only 0 to 2, but the function considers index 8 valid because it trusts the given size. Accessing `a + 8` goes outside the array and dereferencing it can cause undefined behavior because it accesses memory that does not belong to the array.

Step 4 and Step 5: IN LAB:

TASK 3. Pointer Arithmetic & Array Bounds: A Critical Review

Step 1:

Object: Water Bottle

RGB: (R: 40, G: 90, B: 180)

Step2:

```
#include <iostream>
using namespace std;
int main() {
    int rgbVals[3] = {40, 90, 180};
    int* ptr = rgbVals;
    for (int i = 0; i < 3; i++)
        cout << "RGB Value " << i << ": " << *(ptr + i) << endl;
}
```

Step 3:

*** The junior programmer's claim is incorrect. Pointer arithmetic does not automatically perform bounds checking. In C++, `*(arr + i)` and `arr[i]` are essentially equivalent in how they access memory. Neither one is inherently bounds-safe for a raw array.**

*** When `ptr + 3` is calculated, the pointer is moved one position past the last element. This pointer may be formed, but it must not be dereferenced. Therefore `*(ptr + 3)` attempts to access memory outside the array and causes undefined behavior.**

*** A specific unsafe example is:**

```
int arr[3] = {10, 20, 30};
int* ptr = arr;
```

```
cout << *(ptr + 3);
```

Here, the array contains only indices 0, 1, and 2, but `*(ptr + 3)` tries to read index 3. That memory does not belong to the array, so the program has undefined behavior. It might print some value, crash, or appear to work by accident.

Step 4:

The problem becomes more dangerous when an array is passed to a function as a pointer:

```
void printValues(int* ptr) {  
    for (int i = 0; i < 5; i++)  
        cout << *(ptr + i);  
}
```

The function does not know how large the original array is. If the caller passes:

```
int arr[3] = {10, 20, 30};  
printValues(arr);
```

the function still tries to access five elements. The accesses for `i = 3` and `i = 4` go beyond the array, producing undefined behavior.

Therefore, pointer arithmetic is efficient, but it is not automatically safe. The programmer must explicitly track and respect the array's bounds.

Step 5: IN LAB

Step 6:

Reflection: The hardest part was understanding the difference between a pointer being one position past the array and actually accessing that position. At first, I thought `ptr + size` was invalid. Then I understood that creating a one past the end pointer is allowed, but trying to access its value is undefined behavior.

Question 4. Pointer Arithmetic Trace; Edge Case

Pointer Arithmetic Trace and Adversarial Analysis

1. Student-Supplied Data

My chosen word is: SKD

The word SKD represents Skardu, my home city.

2. Numerical Array

S = 19, K = 11, D = 4

Therefore:

```
int data[] = {19, 11, 4};
```

The array length is $N = 3$.

3. Step-by-Step Trace

The function is called as `findMax(&data[0], &data[0] + N)`. Since $N = 3$, start points to offset 0 and end points to offset 3, which is one position past the last element.

Initial state:

`start = offset 0`, so `*start = 19`

`maxVal = 19`

`current = start + 1 = offset 1`

`end = offset 3`

Step	current offset	*current	maxVal before	Condition	maxVal after / explanation
Initial	1	11	19	—	maxVal starts with data[0], which is 19. current starts at data[1].
1	1	11	19	<code>11 > 19</code> is false	11 is smaller than 19, so maxVal stays 19. <code>current++</code> moves it to offset 2.
2	2	4	19	<code>4 > 19</code> is false	4 is smaller than 19, so maxVal stays 19. <code>current++</code> moves it to offset 3.
End	3	—	19	<code>current == end</code>	The loop stops. Offset 3 is one-past-the-end and is not dereferenced.

The function returns 19 because 19 is the largest value in the array.

```
#include <iostream>
```

```
using namespace std;
```

```
int findMax(int* start, int* end) {
```

```
    if (start == end)
```

```
        return -1;
```

```
    int maxVal = *start;
```

```
    int* current = start + 1;
```

```
    while (current != end) {
```

```
        if (*current > maxVal)
```

```
            maxVal = *current;
```

```
        current++;
```

```
}
```

```
return maxVal;
```

```
}
```

```
int main() {
```

```
int data[] = {19, 11, 4};
```

```
int* start = data;
```

```
int* end = data + 3;
```

```
int result = findMax(start, end);
```

```
cout << "Maximum value: " << result << endl;
```

```
return 0;
```

```
}
```

```

1  task4_lab1.cpp > findMax(int*, int*)
2  using namespace std;
3
4  int findMax(int* start, int* end) {
5      if (start == end)
6          return -1;
7
8      int maxVal = *start;
9      int* current = start + 1;
10
11     while (current != end) {
12         if (*current > maxVal)
13             maxVal = *current;
14
15         current++;
16     }
17
18     return maxVal;
19 }
20
21 int main() {
22     int data[] = {19, 11, 4};
23
24     int* start = data;
25     int* end = data + 3;
26
27     int result = findMax(start, end);
28
29     cout << "Maximum value: " << result << endl;
30
31     return 0;
32 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```

ali@fedora:/run/media/ali/New Volume/Work Space/University/FAST/Classes/3rd_Sems/DSA/Classes$ ./task4_lab1
Maximum value: 19
ali@fedora:/run/media/ali/New Volume/Work Space/University/FAST/Classes/3rd_Sems/DSA/Classes$

```

4. Adversarial Analysis

The normal call is safe because `end` is deliberately set to one position past the last element of the same array. However, the function depends completely on the caller providing a correct end pointer. The condition `current != end` only checks whether the two pointer values are exactly equal. It does not by itself check whether `current` is still within the valid array range.

For example, suppose an invalid end pointer refers to a different memory location rather than the one-past-the-end address of data. current would move from offset 1 to offset 2 and then continue beyond the array because it has no reason to become equal to that unrelated end pointer. The loop could then dereference memory outside the array, producing undefined behavior. The important point is that the problem comes from violating the function's expected start/end range, not from pointer arithmetic itself.

5. One-Line Fix

`while (current < end)`

This makes the boundary condition explicit: current must remain before end. This assumes that start and end are valid pointers into the same array and that end is at or after start.

6. Viva Preparation

In my trace, data contains {19, 11, 4}. start is at offset 0 and end is at offset 3. The loop checks offset 1 first, where the value is 11, and then offset 2, where the value is 4. Neither value is greater than the initial maximum of 19. After `current++` at offset 2, current becomes offset 3, which equals end, so the loop stops without dereferencing it.

The main limitation is that `!=` only checks equality with end. If the caller supplies an incorrect end pointer, current can pass the actual array boundary and the next dereference can access invalid memory. Changing the condition to `current < end` gives a direct boundary check when the two pointers belong to the same array.